

5 Project Option 4: Multi-Tenant Data Pipeline

5.1 Project Overview

Build an ETL (Extract, Transform, Load) pipeline system where users can define data pipelines to ingest, transform, and load data. Supports multiple data sources (CSV, JSON, APIs, databases) and transformations.

5.1.1 Key Features Summary

- Phase 1: Pipeline definition, Data ingestion (CSV/JSON), Transformations (filter, map, aggregate), Output to database/file, Status tracking
- Phase 2 Options: Multiple data sources (REST API, Database, S3), Transformation library, Scheduling (cron), Multi-tenancy
- Phase 3 Options: Streaming data, Incremental loading, Error recovery, Pipeline monitoring, Performance optimization

5.1.2 Business Context

- Data pipelines are essential for modern businesses:
- Data Integration: Consolidate data from multiple sources (CRM, ERP, external APIs)
- Data Warehousing: Transform and load data for analytics and reporting
- Data Migration: Move data between systems during upgrades or consolidation
- Real-time Analytics: Process streaming data for dashboards and alerts
- Data Synchronization: Keep multiple systems in sync
- Compliance: Ensure data quality, validation, and audit trails

5.1.3 Real-World Use Cases

E-commerce:

- Daily sales data from multiple stores → Transform → Load into analytics DB
- Product catalog from CSV → Validate and enrich → Update website database

Finance:

- Transaction logs from payment systems → Aggregate → Load into data warehouse
- Market data from APIs → Transform → Store for analysis

Healthcare:

- Patient records from multiple clinics → Normalize → Central database
- Lab results from machines → Parse and validate → EHR system

5.2 Phase 1: Core Implementation (Weeks 1-2)

5.2.1 Pipeline Definition

Users must be able to define pipelines using JSON or YAML configuration files. Each pipeline specifies the source, transformations, and destination.

5.2.2 Pipeline Definition Schema

```
{
  "pipeline_id": "sales_etl_001",
  "pipeline_name": "Daily Sales ETL",
  "tenant_id": "tenant_123",
  "description": "Extract sales data, aggregate by product, load to warehouse",

  "source": {
    "type": "csv",
    "config": {
      "file_path": "/data/sales_2025_02_01.csv",
      "delimiter": ",",
      "has_header": true,
      "encoding": "utf-8"
    },
    "schema": {
      "transaction_id": "string",
      "product_id": "string",
      "quantity": "integer",
      "price": "decimal",
      "timestamp": "datetime"
    }
  },

  "transformations": [
    {
      "type": "filter",
      "config": {
        "condition": "quantity > 0"
      }
    },
    {
      "type": "map",
      "config": {
        "operations": {
          "total_amount": "quantity * price",
          "product_id": "UPPER(product_id)"
        }
      }
    },
    {
      "type": "aggregate",
      "config": {
        "group_by": ["product_id"],
        "aggregations": {
          "total_quantity": "SUM(quantity)",
          "total_revenue": "SUM(total_amount)",
          "transaction_count": "COUNT(*)"
        }
      }
    }
  ],

  "destination": {
```

```

    "type": "database",
    "config": {
      "connection_string": "postgresql://localhost/warehouse",
      "table": "daily_sales_summary",
      "write_mode": "append"
    }
  },

  "execution": {
    "batch_size": 1000,
    "error_handling": "continue",
    "timeout_seconds": 3600
  }
}

```

5.2.3 Required Pipeline Fields

- pipeline_id: Unique identifier (UUID or string)
- pipeline_name: Human-readable name
- tenant_id: Tenant identifier for multi-tenancy
- source: Source configuration with type and config
- transformations: Array of transformation steps (can be empty)
- destination: Destination configuration
- execution: Execution parameters (optional)

5.2.4 Pipeline Validation

- Validate JSON/YAML syntax
- Verify all required fields are present
- Check source type is supported (csv, json for Phase 1)
- Validate transformation types are recognized
- Verify destination is accessible
- Detect circular dependencies (if chaining pipelines)

5.2.5 Data Ingestion

5.2.5.1 CSV Ingestion

- Read CSV files from local filesystem or uploaded files
- Support configurable delimiters (comma, tab, pipe)
- Handle files with/without headers
- Support different encodings (UTF-8, Latin-1)
- Auto-detect data types or use provided schema
- Handle missing values and null values
- Stream large files (don't load entire file into memory)

5.2.5.2 Example CSV Source Config:

```
"source": {
  "type": "csv",
  "config": {
    "file_path": "/uploads/data.csv",
    "delimiter": ",",
    "has_header": true,
    "encoding": "utf-8",
    "skip_rows": 0,
    "null_values": ["", "NULL", "N/A"]
  },
  "schema": {
    "id": "integer",
    "name": "string",
    "amount": "decimal",
    "created_at": "datetime"
  }
}
```

5.2.5.3 JSON Ingestion

- Read JSON files (single object or array of objects)
- Support JSONL (JSON Lines) format
- Handle nested objects (flatten or preserve structure)
- Support JSON Path for extracting data

5.2.5.4 Example JSON Source Config:

```
"source": {
  "type": "json",
  "config": {
    "file_path": "/uploads/data.json",
    "format": "array", // or "jsonl", "object"
    "json_path": "$.data[*]", // optional, extract specific path
    "flatten": false
  }
}
```

5.2.6 Schema Inference

- Automatically detect column types by sampling data
- Infer types: string, integer, decimal, boolean, datetime
- User can override inferred schema

- Validate data against schema during ingestion

5.2.7 Error Handling During Ingestion

- Skip malformed rows and log errors
- Continue processing or fail on first error (configurable)
- Track error count and error rows
- Write failed rows to error log file

DRAFT

5.2.8 Transformation Engine

Implement three core transformation types:

5.2.8.1 Filter Transformation

Remove rows that don't match a condition.

```
{
  "type": "filter",
  "config": {
    "condition": "age >= 18 AND status == 'active'"
  }
}

// Supported operators:
// Comparison: ==, !=, >, <, >=, <=
// Logical: AND, OR, NOT
// String: LIKE, CONTAINS, STARTS_WITH, ENDS_WITH
// Null checks: IS NULL, IS NOT NULL
```

Examples:

- Filter out negative quantities: "quantity > 0"
- Keep only US customers: "country == 'US'"
- Complex: "amount > 1000 AND (status == 'approved' OR vip == true)"

5.2.8.2 Map Transformation

Rename columns, derive new columns, or modify values.

```
{
  "type": "map",
  "config": {
    "operations": {
      "customer_id": "id", // rename
      "full_name": "CONCAT(first_name, ' ', last_name)", // derive
      "price_usd": "price * exchange_rate", // calculate
      "email": "LOWER(email)", // transform
      "category": "UPPER(category)"
    }
  }
}

// Supported functions:
// String: UPPER, LOWER, CONCAT, SUBSTRING, TRIM, REPLACE
// Math: +, -, *, /, ROUND, CEIL, FLOOR, ABS
// Date: YEAR, MONTH, DAY, DATE_ADD, DATE_DIFF
// Conditional: IF, CASE WHEN
```

5.2.8.3 Aggregate Transformation

Group rows by fields and compute aggregates.

```
{
  "type": "aggregate",
  "config": {
    "group_by": ["product_id", "region"],
    "aggregations": {
      "total_quantity": "SUM(quantity)",
      "avg_price": "AVG(price)",
      "order_count": "COUNT(*)",
      "max_amount": "MAX(amount)",
      "min_amount": "MIN(amount)"
    }
  }
}
```

```

    }
  }
}

// Supported aggregations:
// SUM, AVG, COUNT, MIN, MAX, FIRST, LAST

```

5.2.9 Transformation Pipeline

- Transformations are applied in sequence
- Output of one transformation is input to next
- Support any number of transformations
- Each transformation can access previous stage columns

5.2.9.1 Example Transformation Chain:

```

"transformations": [
  {
    "type": "filter",
    "config": { "condition": "status == 'completed'" }
  },
  {
    "type": "map",
    "config": {
      "operations": {
        "revenue": "quantity * unit_price",
        "discount_amount": "revenue * discount_rate"
      }
    }
  },
  {
    "type": "filter",
    "config": { "condition": "revenue > 100" }
  },
  {
    "type": "aggregate",
    "config": {
      "group_by": ["customer_id"],
      "aggregations": {
        "total_revenue": "SUM(revenue)",
        "avg_discount": "AVG(discount_amount)"
      }
    }
  }
]

```

5.2.10 Data Loading (Destination)

5.2.10.1 Database Output

- Write transformed data to relational database
- Support PostgreSQL, MySQL, SQLite
- Create table if it doesn't exist
- Support write modes: append, overwrite, upsert
- Batch inserts for performance
- Transaction support (rollback on failure)

```
"destination": {
  "type": "database",
  "config": {
    "connection_string": "postgresql://user:pass@localhost:5432/warehouse",
    "table": "sales_summary",
    "write_mode": "append", // append, overwrite, upsert
    "batch_size": 1000,
    "upsert_keys": ["product_id", "date"] // for upsert mode
  }
}
```

5.2.10.2 Write Modes:

- append: Add new rows to existing table
- overwrite: Drop table and recreate (TRUNCATE then INSERT)
- upsert: Update existing rows, insert new ones (ON CONFLICT)

5.2.10.3 File Output

- Write transformed data to CSV or JSON file
- Support custom output format
- Compression support (gzip, optional)

```
"destination": {
  "type": "file",
  "config": {
    "output_path": "/outputs/sales_summary.csv",
    "format": "csv", // or "json"
    "delimiter": ",",
    "include_header": true,
    "compression": "gzip" // optional
  }
}
```


5.2.11 Pipeline Execution & Status Tracking

5.2.12 Execution States

Each pipeline execution (job run) goes through these states:

- PENDING: Job created but not started
- RUNNING: Currently executing
- SUCCESS: Completed successfully
- FAILED: Execution failed with errors
- CANCELLED: Manually stopped by user

5.2.13 Execution Metadata

Track detailed execution information:

- job_run_id: Unique ID for this execution
- pipeline_id: Which pipeline was executed
- tenant_id: Tenant who owns this execution
- status: Current state
- start_time: When execution began
- end_time: When execution completed/failed
- duration_seconds: Total execution time
- records_read: Total rows ingested
- records_written: Total rows loaded to destination
- records_filtered: Rows removed by filters
- records_failed: Rows that caused errors
- error_message: Error details if failed
- logs: Execution logs and warnings

5.2.14 Progress Tracking

- Report progress percentage during execution
- Update status in real-time
- Provide estimated time remaining
- Allow users to cancel running jobs

5.2.15 Phase 1 Deliverables

5.2.16 Required Deliverables:

- ✓ Pipeline definition API (create, read, update, delete)
- ✓ CSV and JSON ingestion working
- ✓ Three transformation types: filter, map, aggregate
- ✓ Output to database AND file
- ✓ Pipeline execution with status tracking
- ✓ Error logging for failed rows
- ✓ Database schema implemented
- ✓ All API endpoints working
- ✓ Basic validation and error handling
- ✓ Sample pipeline definitions
- ✓ README with setup instructions
- ✓ Minimum 5 tests

5.2.17 Success Criteria:

- Define pipeline via API → Pipeline stored in database
- Execute pipeline → Data extracted from CSV/JSON
- Apply transformations → Data filtered, mapped, aggregated correctly
- Load to database → Data appears in target table
- Check status → See RUNNING → SUCCESS with row counts
- Handle errors → Failed rows logged, execution continues or fails gracefully

5.3 Phase 2: Feature Extensions (Weeks 3-4)

Choose and implement TWO features:

5.3.1 Option A: Multiple Data Sources

Extend ingestion to support REST APIs, databases, and object storage.

5.3.1.1 Requirements:

- REST API: HTTP GET/POST to external APIs
- Database: Read from PostgreSQL, Oracle, SQLite
- S3-Compatible Storage: Read from MinIO, AWS S3
- Secure credential management
- Connection pooling for databases

5.3.1.2 REST API Source

```
"source": {
  "type": "api",
  "config": {
    "url": "https://api.example.com/data",
    "method": "GET",
    "headers": {
      "Authorization": "Bearer ${API_KEY}",
      "Content-Type": "application/json"
    },
    "params": {
      "date": "2025-02-02",
      "limit": "1000"
    },
    "pagination": {
      "type": "offset", // or "cursor", "page"
      "offset_param": "offset",
      "limit_param": "limit",
      "max_pages": 10
    },
    "json_path": "$.data[*]", // Extract data from response
    "retry": {
      "max_attempts": 3,
      "backoff": "exponential"
    }
  }
}
```

5.3.1.2.1 Implementation:

- Make HTTP requests using REST client
- Handle pagination (fetch multiple pages)
- Parse JSON response
- Extract data using JSON Path
- Retry on failure with exponential backoff
- Rate limiting (respect API limits)

5.3.1.3 Database Source

```
"source": {
  "type": "database",
  "config": {
```

```
"connection": {
  "driver": "postgresql",
  "host": "db.example.com",
  "port": 5432,
  "database": "production",
  "username": "etl_user",
  "password": "${DB_PASSWORD}"
},
"query": "SELECT * FROM orders WHERE created_at >= '2025-02-01'",
"fetch_size": 1000 // Batch size for streaming
}
```

5.3.1.3.1 Implementation:

- Establish database connection
- Execute SQL query
- Stream results in batches (don't load all into memory)
- Support prepared statements for safety
- Connection pooling for efficiency

5.3.1.4 S3 / Object Storage Source

```
"source": {
  "type": "s3",
  "config": {
    "endpoint": "https://s3.amazonaws.com", // or MinIO URL
    "bucket": "data-lake",
    "key": "sales/2025/02/01/data.csv", // or prefix for multiple files
    "access_key": "${S3_ACCESS_KEY}",
    "secret_key": "${S3_SECRET_KEY}",
    "region": "us-east-1",
    "format": "csv"
  }
}
```

5.3.1.4.1 Implementation:

- Connect to S3-compatible storage
- Download object(s)
- Parse file based on format (CSV, JSON)
- Support reading multiple files by prefix
- Stream large files

5.3.1.5 Credential Management

- Store credentials securely (NOT in pipeline definition)
- Use environment variables: \${VARIABLE_NAME}
- Or use secret management system (HashiCorp Vault, AWS Secrets Manager)
- Encrypt sensitive data in database

5.3.1.6 Deliverables:

- At least TWO additional source types implemented
- Secure credential handling
- Connection pooling (for database sources)

- Tests for each source type
- Documentation with examples

DRAFT

5.3.2 Option B: Transformation Library (Pluggable Components)

Design transformations as pluggable components so new operations can be added without modifying core engine.

5.3.2.1 Requirements:

- Abstract transformation interface/base class
- Each transformation is independent module
- Registration system to discover transformations
- Easy to add new transformations
- Transformations can be chained

5.3.2.2 Additional Transformations to Implement

1. Sort Transformation:

```
{
  "type": "sort",
  "config": {
    "columns": ["amount DESC", "date ASC"]
  }
}
```

2. Deduplicate Transformation:

```
{
  "type": "deduplicate",
  "config": {
    "columns": ["customer_id", "email"],
    "keep": "first" // or "last"
  }
}
```

3. Join Transformation:

```
{
  "type": "join",
  "config": {
    "right_source": {
      "type": "csv",
      "config": { "file_path": "/data/customers.csv" }
    },
    "join_type": "left", // inner, left, right, full
    "on": ["customer_id"]
  }
}
```

4. Pivot Transformation:

```
{
  "type": "pivot",
  "config": {
    "index": ["date"],
    "columns": ["product_id"],
    "values": "quantity",
    "aggregation": "sum"
  }
}
```

5. Validation Transformation:

```
{
  "type": "validate",
```

```
"config": {
  "rules": [
    {"column": "email", "regex":
"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"},
    {"column": "age", "min": 0, "max": 150},
    {"column": "country", "allowed_values": ["US", "UK", "CA"]}
  ],
  "on_error": "skip" // or "fail"
}
```

5.3.2.3 Deliverables:

- Transformation interface/base class
- Registration system
- At least 3 additional transformations beyond filter/map/aggregate
- Documentation for creating new transformations
- Tests for each transformation

5.3.3 Option C: Scheduling System (Cron-based)

Allow pipelines to run automatically on a schedule.

5.3.3.1 Requirements:

- Define schedule using cron expressions
- Trigger pipeline executions automatically
- Prevent overlapping runs (skip or queue)
- Track scheduled vs manual executions
- Enable/disable schedules

5.3.3.2 Schedule Definition

```
{
  "pipeline_id": "daily_sales_etl",
  "schedule": {
    "enabled": true,
    "cron": "0 2 * * *", // Every day at 2 AM
    "timezone": "America/New_York",
    "overlap_policy": "skip", // skip, queue, cancel_previous
    "max_retries": 3,
    "retry_delay_minutes": 5
  }
}

// Common cron expressions:
// "0 * * * *" - Every hour
// "0 */6 * * *" - Every 6 hours
// "0 0 * * *" - Daily at midnight
// "0 0 * * 0" - Weekly on Sunday
// "0 0 1 * * *" - Monthly on 1st
```

5.3.3.3 Overlap Policies

- skip: Don't run if previous execution still running
- queue: Queue new run, execute when previous finishes
- cancel_previous: Stop previous run and start new one

5.3.3.4 Deliverables:

- Cron expression parser and scheduler
- Schedule management APIs
- Overlap policy implementation
- Background scheduler service
- Schedule history tracking
- Tests for scheduling logic
- Documentation with cron examples

5.3.4 Option D: Multi-Tenancy (Strict Isolation)

Ensure complete tenant isolation at data, configuration, and execution levels.

5.3.4.1 Requirements:

- Tenant identification and authentication
- Data isolation: One tenant cannot access another tenant's data
- Pipeline isolation: Each tenant has own pipelines
- Execution isolation: Jobs run in isolated contexts
- Resource quotas per tenant (optional)

5.3.4.2 Execution Isolation

- Each pipeline execution runs with tenant context
- Temporary files stored in tenant-specific directories
- Output files/tables prefixed with tenant ID
- Logs include tenant ID for auditing

5.3.4.3 Testing Multi-Tenancy

- Create two tenants with different API keys
- Tenant A creates pipeline → Tenant B cannot see it
- Tenant A executes job → Tenant B cannot access job details
- Attempt to access another tenant's resources → 403 Forbidden
- Test resource quotas

5.3.4.4 Deliverables:

- Tenant model and authentication
- Complete data isolation
- Tenant context in all operations
- Resource quota enforcement
- Security tests for tenant isolation
- Documentation on multi-tenancy

5.4 Phase 3: Advanced Features (Weeks 5-6)

Choose ONE advanced feature:

5.4.1 Option A: Streaming Data (Real-time Processing)

Process data in real-time from streaming sources like Kafka.

5.4.1.1 Requirements:

- Connect to Apache Kafka or similar message broker
- Consume messages in real-time
- Apply transformations to streaming data
- Write to destination continuously
- Handle backpressure

5.4.1.2 Implementation:

```
"source": {
  "type": "kafka",
  "config": {
    "bootstrap_servers": "localhost:9092",
    "topic": "sales_events",
    "group_id": "etl_consumer_group",
    "auto_offset_reset": "earliest",
    "max_poll_records": 500
  }
}

// Consumer loop
while (running) {
  ConsumerRecords<String, String> records =
  consumer.poll(Duration.ofMillis(1000));

  List<Record> batch = new ArrayList<>();
  for (ConsumerRecord<String, String> record : records) {
    batch.add(parseRecord(record.value()));

    if (batch.size() >= batchSize) {
      processBatch(batch);
      batch.clear();
    }
  }

  if (!batch.isEmpty()) {
    processBatch(batch);
  }

  consumer.commitSync();
}
```

5.4.1.3 Deliverables:

- Kafka integration
- Micro-batching for efficiency
- Offset management
- Error handling in streams
- Tests with embedded Kafka

5.4.2 Option B: Incremental Loading (Change Data Capture)

Only process new or changed records instead of full reloads.

5.4.2.1 Requirements:

- Track last processed timestamp or ID
- Load only new records since last run
- Support upsert for changed records
- Checkpointing for failure recovery

5.4.2.2 Implementation:

```
// Watermark/Checkpoint tracking
CREATE TABLE pipeline_checkpoints (
  pipeline_id VARCHAR(100) PRIMARY KEY,
  last_processed_timestamp TIMESTAMP,
  last_processed_id BIGINT,
  updated_at TIMESTAMP DEFAULT NOW()
);

// Pipeline reads from checkpoint
"source": {
  "type": "database",
  "config": {
    "query": "SELECT * FROM orders WHERE updated_at > ${CHECKPOINT_TIMESTAMP}",
    "incremental": {
      "enabled": true,
      "watermark_column": "updated_at",
      "checkpoint_column": "id"
    }
  }
}

// After successful load, update checkpoint
UPDATE pipeline_checkpoints
SET last_processed_timestamp = :maxTimestamp,
    last_processed_id = :maxId
WHERE pipeline_id = :pipelineId;
```

5.4.2.3 Deliverables:

- Checkpoint tracking
- Incremental queries
- Upsert support
- Tests for incremental loads

5.4.3 Option C: Error Recovery & Dead Letter Queue

Advanced error handling with retries and dead letter queue.

5.4.3.1 Requirements:

- Automatic retry on transient failures
- Dead Letter Queue for permanent failures
- Manual replay from DLQ
- Error categorization

5.4.3.2 Implementation:

```
CREATE TABLE dead_letter_queue (
  dlq_id SERIAL PRIMARY KEY,
  job_run_id UUID,
  row_number BIGINT,
  error_type VARCHAR(50),
  error_message TEXT,
  row_data JSONB,
  retry_count INTEGER DEFAULT 0,
  max_retries INTEGER DEFAULT 3,
  created_at TIMESTAMP DEFAULT NOW(),
  last_retry_at TIMESTAMP,
  status VARCHAR(20) DEFAULT 'FAILED'
);

// Retry logic
for (int attempt = 0; attempt <= maxRetries; attempt++) {
  try {
    processRow(row);
    break;
  } catch (TransientException e) {
    if (attempt < maxRetries) {
      Thread.sleep((long) Math.pow(2, attempt) * 1000);
      continue;
    }
    // Max retries exceeded
    sendToDLQ(row, e);
  } catch (PermanentException e) {
    sendToDLQ(row, e);
    break;
  }
}
```

5.4.3.3 Deliverables:

- Retry mechanism with exponential backoff
- DLQ table and APIs
- Manual replay capability
- Tests for error scenarios

5.4.4 Option D: Pipeline Monitoring & Metrics

Comprehensive monitoring with metrics, alerts, and dashboards.

5.4.4.1 Requirements:

- Collect execution metrics
- Real-time dashboard
- Alert on failures or SLA violations
- Historical trends

5.4.4.2 Metrics to Track:

- Execution time per pipeline
- Success/failure rate
- Records processed per minute
- Error rate
- Resource utilization

5.4.4.3 Implementation:

```
// Prometheus metrics
pipeline_executions_total{pipeline="sales_etl", status="success"} 125
pipeline_executions_total{pipeline="sales_etl", status="failed"} 2
pipeline_execution_duration_seconds{pipeline="sales_etl"} 45.3
pipeline_records_processed{pipeline="sales_etl"} 10000

// Alert rules
ALERT PipelineFailureRate
  IF rate(pipeline_executions_total{status="failed"}[5m]) > 0.1
  FOR 5m
  ANNOTATIONS {
    summary = "High pipeline failure rate"
  }
```

5.4.4.4 Deliverables:

- Metrics collection
- Prometheus endpoint or dashboard
- Alert mechanism
- Visualization

5.4.5 Option E: Performance Optimization

Optimize for large datasets (millions of rows).

5.4.5.1 Optimization Techniques:

- Parallel processing: Split data into chunks, process concurrently
- Memory management: Stream data, don't load all into memory
- Batch operations: Bulk inserts instead of row-by-row
- Indexing: Create indexes on destination tables
- Connection pooling: Reuse database connections
- Caching: Cache lookups for map operations

5.4.5.2 Benchmarks:

- Process 1 million rows in under 2 minutes
- Memory usage stays under 512 MB
- CPU utilization > 70% (good parallelization)

5.4.5.3 Deliverables:

- Performance optimizations implemented
- Benchmark results
- Load testing scripts
- Performance documentation